NestShip Unified Project Roadmap

- Product: NestShip commercial SaaS boilerplate
- Architecture: Separate `nestship-backend` and `nestship-frontend` repositories
- Backend stack: NestJS 11, Node.js 24, pnpm 11, PostgreSQL, and Prisma
- Frontend stack: React 19, Vite 8, TypeScript, npm, Tailwind CSS, shadcn, Base UI, and React Router
- API contract: Versioned REST API and Swagger/OpenAPI under `/api/v1`
- Roadmap baseline: 2026-08-11
- Branch policy: Work directly on `main` unless the team changes this policy

Purpose

This is the authoritative implementation roadmap for the complete NestShip product. It replaces separate frontend and backend planning documents with one phase sequence that shows both sides of each product capability.

A phase is marked Complete only when all required backend, frontend, contract, test, and documentation work assigned to that phase is complete. Backend-only foundation phases can be Complete before later frontend integration phases.

Status Legend

Status	Meaning
Complete	All required work for the phase is implemented and verified
In progress	Useful work exists, but one or more required deliverables remain
Ready	Dependencies are available and implementation can begin
Pending	Required implementation has not started
Deferred	Work is deliberately postponed to a named phase or decision gate

Project Snapshot

Backend

1. Phases 1 through 5, 7 through 10, 12, and 13 are complete.
2. Phase 14 has a complete provider-independent production baseline.
3. Provider-dependent Phase 14 decisions and Phase 15 CI/CD remain.
4. The backend exposes authentication, organizations, permissions, email, billing, notifications, and file-storage contracts.

Frontend

1. Phase 1 repository foundations are complete and use npm 11 with Node.js 24.
2. React, Vite, strict TypeScript, Tailwind, shadcn, Base UI, Lucide, React Router, design tokens, and a responsive admin shell are present.
3. The frontend has validated environment configuration, a centralized native fetch client, TanStack Query, memory-only Zustand stores, and generated OpenAPI contract types.
4. Vitest, React Testing Library, MSW, and Playwright smoke coverage pass, and the development UI reports backend readiness.
5. Phase 6 browser authentication is complete with session restoration, protected routes, verification, password recovery, OAuth callback handling, logout, and logout-all.
6. Dashboard, Team, Billing, and Notification Preferences screens remain primarily mock or local-state UI pending feature integration.

Phase Summary

Phase	Product scope	Backend status	Frontend status	Overall status
Phase 0	Architecture and product decisions	In progress	Complete	In progress
Phase 1	Repository foundations	Complete	Complete	Complete
Phase 2	Local backend infrastructure	Complete	Not applicable	Complete
Phase 3	Backend application foundation	Complete	Not applicable	Complete
Phase 4	Identity and tenancy database foundation	Complete	Not applicable	Complete
Phase 5	Backend authentication	Complete	Not applicable	Complete
Phase 6	Frontend authentication	Complete	Complete	Complete
Phase 7	Organizations and memberships	Complete	Mock UI partially present	In progress
Phase 8	Permission-aware behavior	Complete	Demo gating partially present	In progress
Phase 9	Transactional email and browser link journeys	Complete	Pending	In progress
Phase 10	Billing and entitlements	Complete	Mock UI partially present	In progress
Phase 11	Dashboard and settings experience	Not applicable	UI partially present	In progress
Phase 12	Notifications	Complete	Preferences mock partially present	In progress
Phase 13	File management	Complete	Pending	In progress
Phase 14	Production hardening	In progress	Pending	In progress
Phase 15	CI/CD, regression testing, and documentation	Pending	Pending	Pending
Release	Beta, packaging, and commercial launch	Pending	Pending	Pending

Phase 0: Architecture And Product Decisions

Status: In progress

Objective: Establish shared technical, security, product, ownership, and release rules before dependent implementation needs to invent them.

Completed Decisions

1. Use separate frontend and backend repositories with independent deployment.
2. Use NestJS, PostgreSQL, Prisma, and a versioned REST API for the backend.
3. Use React, Vite, TypeScript, React Router, Tailwind CSS, shadcn, Base UI, and Lucide for the frontend.
4. Use Swagger/OpenAPI as the initial frontend-to-backend contract.
5. Keep browser access tokens in memory and refresh tokens in HttpOnly cookies.
6. Use explicit organization IDs for tenant-scoped routes.
7. Keep backend authorization, membership, entitlements, and tenant isolation authoritative.
8. Use Owner, Admin, Member, and Viewer organization roles.
9. Use local file storage in development and private AWS S3 storage in production.
10. Keep Redis out of the runtime until it has an approved responsibility.
11. Use npm for the current frontend repository and pnpm for the backend.
12. Use TanStack Query for server state and Zustand only for memory-only access token and active organization state.
13. Use native `fetch` behind a centralized API client and React Hook Form with Zod for forms and client validation.
14. Generate committed TypeScript contract types from backend OpenAPI with `@hey-api/openapi-ts`.
15. Use Vitest, React Testing Library, MSW, and Playwright for frontend testing.
16. Support current Chrome, Edge, Firefox, and Safari releases with responsive layouts from 360 CSS pixels upward.
17. Regenerate frontend API types whenever the backend contract changes before related frontend integration is merged.

Remaining Decisions

1. Select the production deployment platform and managed PostgreSQL topology.
2. Select centralized logging, error monitoring, alerting, and uptime services.
3. Decide whether the first release needs Redis, horizontal scaling, or distributed rate limiting.
4. Define branch protection, release ownership, semantic versioning, support, and coordinated release rules.
5. Decide frontend analytics, privacy, consent, preview deployment, and source map policies.

Recommended Frontend Baseline

1. Use TanStack Query for server-owned state.
2. Use Zustand only for in-memory authentication and active organization state.
3. Use native `fetch` behind a centralized typed API client.
4. Use React Hook Form and Zod for form state and client validation.

5. Generate TypeScript types from the backend OpenAPI document.
6. Use Vitest, React Testing Library, MSW, and Playwright.

Completion Criteria

1. Every remaining decision has an ADR or explicit deferral.
2. No implementation phase must invent security or compatibility rules.

Phase 1: Repository Foundations

Status: Complete

Objective: Make both repositories independently installable, verifiable, documented, and ready for feature work.

Backend - Complete

1. Created and pushed the NestJS repository.
2. Enabled strict TypeScript and pinned Node.js 24 and pnpm 11 expectations.
3. Added format, lint, unit, e2e, build, Prisma, and development scripts.
4. Added environment templates, LF line endings, and secret exclusions.
5. Established direct-to-main workflow for the current team.

Frontend - Completed

1. Created and cloned the frontend repository with a Git remote and `main`.
2. Scaffolded React 19, Vite 8, and TypeScript with npm.
3. Added the `@/` import alias.
4. Added Tailwind CSS, shadcn, Base UI, design tokens, fonts, and Lucide.
5. Added React Router, a responsive admin layout, sidebar, top bar, and routes.
6. Added development, lint, build, and preview scripts.
7. Installed dependencies successfully with no reported npm vulnerabilities.
8. Corrected the cloned lint and TypeScript build blockers.
9. Declared Node.js 24 and npm 11 and enabled strict TypeScript.
10. Added formatting, explicit type-check, unit-test, browser-test, and OpenAPI generation scripts.
11. Added `.env.example` and validated API/public URL configuration.
12. Added the centralized API client with JSON, bearer token, credentialed cookie, standard error, and request-ID behavior.
13. Added TanStack Query and memory-only Zustand authentication and active organization stores.
14. Generated committed TypeScript contract types from backend OpenAPI.
15. Added Vitest, Testing Library, MSW, Playwright, and initial smoke tests.
16. Replaced the Vite template README with setup, environment, security, and architecture documentation.
17. Added the line-ending policy and confirmed the local origin against backend CORS.
18. Added a development-only controlled readiness result from the backend.

Completion Criteria

1. Clean checkouts of both repositories install and pass their quality scripts.
2. No local credentials or environment files are committed.
3. The frontend displays a controlled result from the backend health API.

Phase 2: Local Backend Infrastructure

Status: Complete

Objective: Provide repeatable local infrastructure for backend development.

Delivered

1. Added PostgreSQL 18.4 through Docker Compose.
2. Bound it to loopback with development-only credentials.
3. Added health checks, persistence, networking, and documented recovery.
4. Verified startup, readiness, persistence, restart, backup, and restore.
5. Deferred Redis until a production responsibility is approved.

Completion Criteria

1. PostgreSQL starts healthy and retains data across container recreation.
2. The backend connects through documented local configuration.

Phase 3: Backend Application Foundation

Status: Complete

Objective: Establish behavior shared by every backend module.

Delivered

1. Added validated configuration, `/api/v1`, URI versioning, and strict DTO validation.
2. Added standard errors, request IDs, CORS, Swagger/OpenAPI, and Prisma.
3. Added liveness, database readiness, request logging, and graceful shutdown.
4. Added unit and e2e coverage for startup and API conventions.

Completion Criteria

1. Readiness verifies configuration and PostgreSQL.
2. Swagger publishes the versioned contract.
3. Format, lint, tests, e2e tests, and build pass.

Phase 4: Identity And Tenancy Database Foundation

Status: Complete

Objective: Establish durable identity, session, organization, membership, and invitation data boundaries.

Delivered

1. Added User, OAuthAccount, Session, token, Organization, Membership, and Invitation models.
2. Added UUID v7 identifiers, PostgreSQL-native types, constraints, and indexes.
3. Enforced normalized emails, ownership, pending-invitation, and refresh-family rules.
4. Added migrations, idempotent seed data, database contract tests, and ADRs.
5. Deferred billing, notification, and file records to their owning phases.

Completion Criteria

1. Migrations and seeds are repeatable.
2. Database constraints protect authentication and tenant invariants.

Phase 5: Backend Authentication

Status: Complete

Objective: Deliver the secure backend account lifecycle required by browser authentication.

Delivered

1. Added registration, login, current user, refresh rotation, replay detection, logout, and logout-all.
2. Added Argon2id password hashing, short-lived access tokens, and opaque refresh cookies.
3. Added email verification, forgot/reset password, and reset-time session revocation.
4. Added origin checks, rate limits, Google/GitHub OAuth boundaries, Swagger, tests, and security ADRs.

External Configuration

1. Google and GitHub credentials are needed only to enable those OAuth options.
2. Production email credentials and DNS are covered by Phase 9 deployment.

Completion Criteria

1. The full backend authentication lifecycle is tested.
2. Passwords and raw persistent tokens never enter API responses or storage.

Phase 6: Frontend Authentication

Status: Complete

Objective: Deliver the first complete browser workflow against the backend authentication API.

Backend Contract - Complete

1. Registration, login, current user, refresh, logout, and logout-all.
2. Verification, forgot/reset password, and optional OAuth endpoints.
3. Standard errors, throttling responses, and refresh-cookie semantics.

Frontend - Complete

1. Added public-only, verification, OAuth callback, and protected route groups.
2. Added in-memory access-token and current-user state with startup session restoration through the HttpOnly refresh cookie.
3. Added one coordinated refresh attempt for simultaneous `401` responses and prevented stale protected content during restoration or failure.
4. Built registration, login, verification, forgot-password, reset-password, logout, and logout-all journeys.
5. Added current-user loading, protected transitions, OAuth feature flags, and callback handling.
6. Added accessible validation, loading, throttled, expired, offline, and server error states.
7. Added unit, integration, and Playwright authentication coverage.
8. Replaced mock shell identity and unsupported password actions with live backend-supported account and session behavior.

Security Requirements

1. Never persist access tokens in browser storage.
2. Never read or expose the HttpOnly refresh cookie.
3. Never place access or refresh tokens in URLs.
4. Prevent refresh loops and stale protected content.

Completion Criteria

1. Users can register, restore sessions, authenticate, verify, reset passwords, access protected routes, and sign out.
2. Browser tests cover happy paths and high-risk failures.

Phase 7: Organizations And Memberships

Status: In progress

Objective: Support complete multi-organization lifecycle and membership management.

Backend - Complete

1. Added organization CRUD, stable slugs, listing, and soft deletion.
2. Added invitations, membership management, leave, removal, and ownership transfer.
3. Added explicit organization context, pagination, transactions, and exhaustive tenant-isolation tests.

Frontend - Partially Present

1. Team member, pending invitation, role, search, pagination, confirmation, and empty-state UI exists.
2. Current team and organization data is mock data.
3. Role changes, invitations, and removals only mutate local React state.

Frontend - Remaining

1. Add organization creation, listing, switching, and settings routes.
2. Validate active organization state against current memberships.
3. Connect invitation creation, acceptance, resend, revoke, and expiry states.
4. Connect member role, removal, leave, ownership transfer, and deletion flows.
5. Handle membership loss and tenant changes without stale data.
6. Add API, route, and browser tests for high-risk organization workflows.

Completion Criteria

1. Tenant switching never shows stale data from another organization.
2. Invitations and memberships reflect backend state.

Phase 8: Permission-Aware Behavior

Status: In progress

Objective: Apply the Owner, Admin, Member, and Viewer capability matrix in both enforcement and presentation.

Backend - Complete

1. Added central capabilities, decorators, guards, and authorization services.
2. Protected restricted tenant reads and mutations.
3. Added table-driven allow and deny coverage for every role.

Frontend - Partially Present

1. Team and Billing screens contain demo role checks and permission-denied UI.
2. Current checks use local role names and are not connected to backend capabilities.

Frontend - Remaining

1. Consume the approved capability contract through central helpers.
2. Protect navigation and organization, billing, notification, and file actions.
3. Recover safely when capabilities change after render.
4. Add role-matrix component and route tests.

Completion Criteria

1. Visible actions match backend capabilities.
2. The frontend never becomes an authorization boundary.

Phase 9: Transactional Email And Browser Link Journeys

Status: In progress

Objective: Deliver provider-neutral transactional email and safe browser handling for every emailed action.

Backend - Complete

1. Added provider-neutral email interfaces with Resend, local log, and capture adapters.
2. Added verification, reset, invitation, and security templates.
3. Added metadata persistence, retries, idempotency, webhook verification, and tests without requiring a live provider.

Frontend - Remaining

1. Add verification, password reset, invitation acceptance, and recovery routes.
2. Consume URL tokens only on their owning route and remove them from history.
3. Add success, expired, reused, revoked, and malformed states.
4. Preserve invitation destinations safely through authentication.
5. Test local, preview, staging, production, responsive, and accessible journeys.

Completion Criteria

1. Every email action reaches the correct frontend and backend workflow.
2. Sensitive tokens do not remain in browser history.

Phase 10: Billing And Entitlements

Status: In progress

Objective: Provide reliable organization billing, subscription, and entitlement workflows.

Backend - Complete

1. Added Free and Pro plan rules, trials, grace periods, cancellation, refunds, Stripe Tax, and local entitlements.
2. Added Stripe and local adapters, billing models, Checkout, Portal, webhooks, deduplication, ordering protection, and tests.

Frontend - Partially Present

1. Current plan, plan features, payment method, invoice history, annual toggle, pagination, and billing permission UI exists.
2. All billing content is mock data and no control calls the backend.

Frontend - Remaining

1. Load plan catalog and organization billing summary.
2. Connect Checkout and Customer Portal redirects through backend-created URLs.
3. Add trial, active, cancellation, past-due, grace, canceled, incomplete, and delayed-webhook states.
4. Apply backend capabilities and entitlements to controls and upgrade prompts.
5. Add contract, redirect-boundary, error, and browser tests.

Completion Criteria

1. Authorized users can subscribe and manage billing through backend sessions.
2. The UI reflects local backend billing state without duplicating billing truth.

Phase 11: Dashboard And Settings Experience

Status: In progress

Objective: Turn product capabilities into an efficient, responsive, and accessible SaaS interface.

Frontend - Partially Present

1. Responsive shell, sidebar, top bar, mobile navigation, and route redirects exist.
2. Dashboard cards, charts, members table, pagination, and organization/user presentation exist using mock data.
3. Profile, Password, Team, Billing, and Notification Preferences screens exist.
4. Reusable buttons, inputs, cards, tables, tabs, dialogs, menus, badges, skeletons, switches, selects, avatars, and separators exist.

Frontend - Remaining

1. Replace mock user, organization, team, billing, and dashboard data.
2. Connect Profile, Password, and settings controls to real contracts.
3. Add onboarding and first-organization journeys.
4. Add complete loading, empty, stale, offline, and error states.
5. Add accessible organization switching, focus behavior, tooltips, toasts, and confirmations.
6. Audit responsiveness, text containment, keyboard access, contrast, and reduced motion.
7. Add route-level code splitting; the current build reports a large main chunk.

Completion Criteria

1. Core workflows are efficient on supported desktop and mobile viewports.
2. Settings surfaces use real backend contracts.
3. Reusable patterns meet documented accessibility standards.

Phase 12: Notifications

Status: In progress

Objective: Provide persistent user notifications and preferences.

Backend - Complete

1. Added persistent Security, Organization, and Billing notifications.
2. Added cursor pagination, unread counts, mark-read, mark-all-read, preferences, tenant safety, retention, and domain-event tests.

Frontend - Partially Present

1. A notification-preferences screen and top-bar notification indicator exist.
2. Preferences are static controls and the indicator uses no backend count.

Frontend - Remaining

1. Add notification menu, full list, unread count, mark-read, and mark-all-read.
2. Connect organization filters, cursor loading, and preferences.
3. Add visibility-aware polling and safe deep links.
4. Add loading, empty, offline, retry, tenant-loss, and expired-destination states.
5. Add synchronization, pagination, preference, and switching tests.

Completion Criteria

1. Users manage notifications without duplicate, stale, or cross-tenant data.
2. Notification links respect current membership and permissions.

Phase 13: File Management

Status: In progress

Objective: Provide secure organization-scoped upload, download, recovery, and deletion workflows.

Backend - Complete

1. Added provider-neutral file storage contracts and metadata lifecycle.
2. Added local development storage and private AWS S3 production storage.
3. Added quotas, signed transfers, type and size validation, malware-state policy, recovery, purge, orphan cleanup, authorization, and tests.

Frontend - Remaining

1. Add file upload controls, validation, progress, cancellation, and retries.
2. Add organization file listing, metadata, pagination, and usage display.
3. Request short-lived download URLs only when needed.
4. Add permission-aware deletion, restore, and rollback behavior.
5. Handle expired URLs, failed uploads, lost permissions, and cleanup states.
6. Add component, API, and browser tests.

Completion Criteria

1. Authorized users can upload, retrieve, restore, and delete tenant files.
2. Provider credentials, object keys, and permanent private URLs never enter UI code.

Phase 14: Production Hardening

Status: In progress

Objective: Meet measurable security, reliability, accessibility, performance, deployment, and operational standards.

Backend - Provider-Independent Baseline Complete

1. Added structured redacted JSON logging and request tracing.
2. Added security headers, strict CORS, trusted-proxy configuration, body limits, timeouts, and configurable throttling.
3. Added PostgreSQL statement timeouts and parameter-free slow-query warnings.
4. Added non-root runtime and migration Docker images with health checks.
5. Added graceful shutdown, backup/restore tools, and deployment, rollback, backup, restore, incident, and acceptance documentation.
6. Verified unit, e2e, build, lint, schema, migration, container, and production dependency checks.

Backend - Remaining

1. Select deployment, managed PostgreSQL, logging, monitoring, alerting, uptime, and off-site backup providers.
2. Decide whether horizontal scaling and distributed throttling are required.
3. Validate the final production topology and record acceptance evidence.

Frontend - Remaining

1. Add route error boundaries, monitoring boundaries, and release tagging.
2. Review token, credential, redirect, URL-token, and sensitive-log behavior.
3. Define CSP compatibility and source-map policy.
4. Add route splitting and measure loading, bundle, and interaction performance.
5. Run accessibility, keyboard, responsive, and supported-browser audits.
6. Validate production API, CORS, cookie, cache, rollback, and incident behavior.

Completion Criteria

1. Both applications meet approved security and reliability checks.
2. Supported workflows meet browser and accessibility targets.
3. Deployments are observable, recoverable, and documented.

Phase 15: CI/CD, Regression Testing, And Documentation

Status: Pending

Objective: Make product delivery repeatable, verifiable, maintainable, and buyer-ready.

Backend - Remaining

1. Add CI for frozen install, formatting, lint, unit/e2e tests, Prisma validation, migration checks, audit, and build.
2. Add an isolated PostgreSQL CI service and broad regression coverage.
3. Publish versioned OpenAPI artifacts and decide generated-client ownership.
4. Document environments, migrations, providers, deployment, upgrades, and troubleshooting.

Frontend - Remaining

1. Add CI for frozen npm install, formatting, lint, type checks, unit tests, build, and Playwright.
2. Add authentication, invitation, tenant, capability, billing, notification, and file regression coverage.
3. Add approved visual regression and preview deployment checks.
4. Generate or version API types from the OpenAPI contract.
5. Document architecture, state ownership, API use, testing, accessibility, deployment, extension, and troubleshooting.

Shared Release Work

1. Define semantic versions, tags, changelogs, compatibility, and support.
2. Add dependency-update automation with required verification.
3. Declare compatible frontend and backend release versions.

Completion Criteria

1. Every pull request and release runs required checks automatically.
2. New contributors can configure, test, deploy, and extend both repositories.

Release: Beta, Packaging, And Commercial Launch

Status: Pending

Objective: Ship a reproducible, documented, supportable NestShip v1 product.

Ordered Work

1. Freeze the API, database migration, frontend contract, and compatibility baseline.
2. Create realistic non-sensitive demo data and a resettable demo environment.
3. Validate complete account, organization, billing, notification, and file journeys with beta users.
4. Resolve release-blocking security, accessibility, browser, responsive, and workflow defects.

5. Prepare screenshots, demonstrations, product documentation, repository presentation, and launch assets from real working states.
6. Publish versioned release notes, migration guidance, support policy, and source/update delivery procedures.

Completion Criteria

1. The release is reproducible from clean checkouts.
2. Core buyer and user workflows operate end to end.
3. Upgrade and rollback paths are documented.
4. No release-blocking defects remain.

Shared Frontend-To-Backend Contract

1. Swagger/OpenAPI is the authoritative initial HTTP contract.
2. All endpoints remain under `/api/v1` until a coordinated version change.
3. The frontend keeps access tokens in memory and uses credentialed refresh and logout requests.
4. The refresh token remains inaccessible to frontend JavaScript.
5. The frontend maps standard backend errors and request IDs into safe user and support messages.
6. Tenant-scoped routes and cache keys use explicit organization IDs.
7. Membership, permissions, entitlements, and tenant isolation remain backend responsibilities.
8. Capability and entitlement data is refreshed after relevant mutations.
9. Breaking changes require versioned artifacts and compatibility notes.

Immediate Next Work

1. Implement Phase 7 frontend organization and membership integration.
2. Replace mock permission, billing, dashboard, notification, and file surfaces in dependency order.
3. Complete provider-dependent Phase 14 backend decisions before production.
4. Implement shared Phase 15 CI/CD and release documentation after feature integration stabilizes.